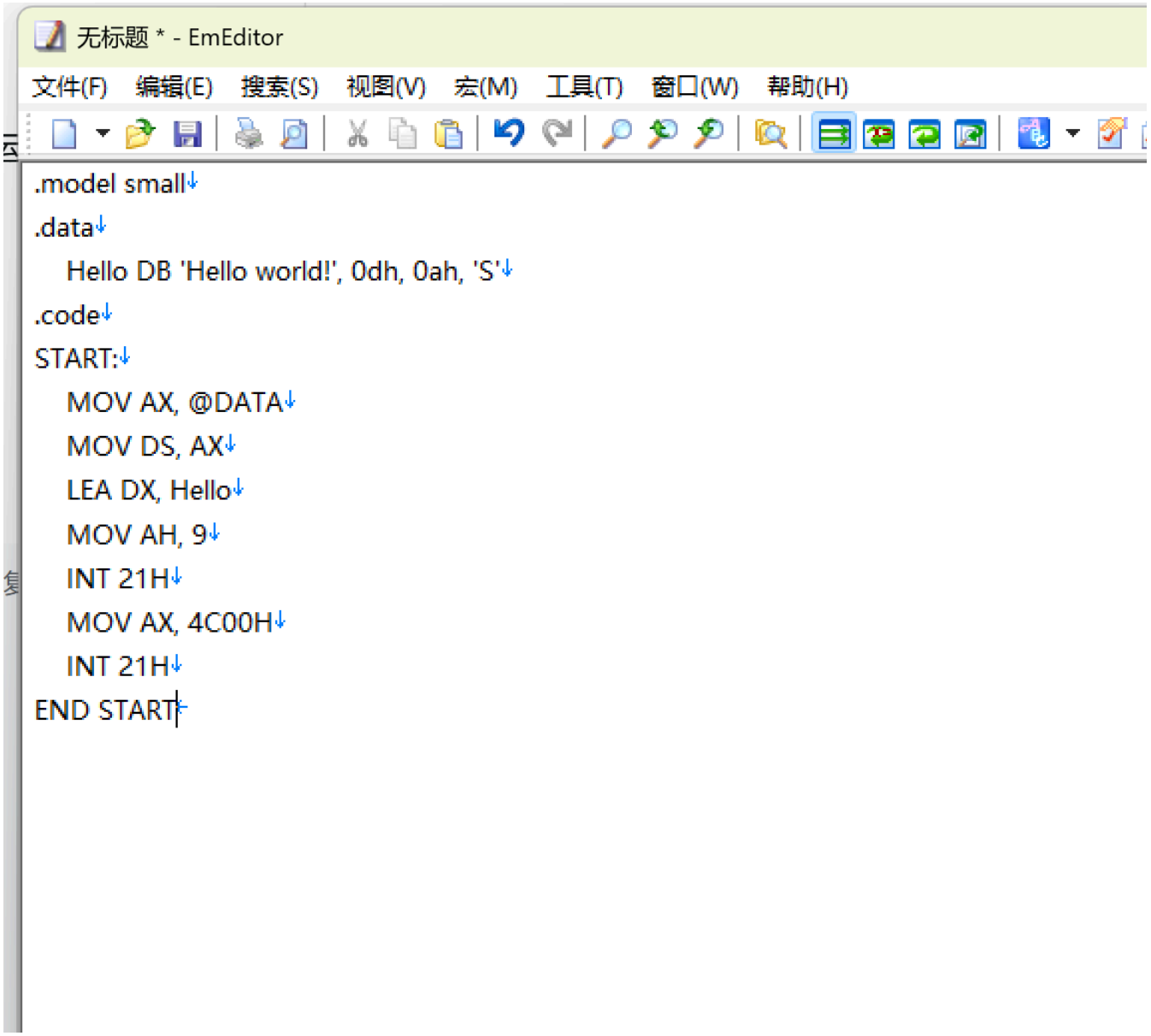
汇编语言第一次作业

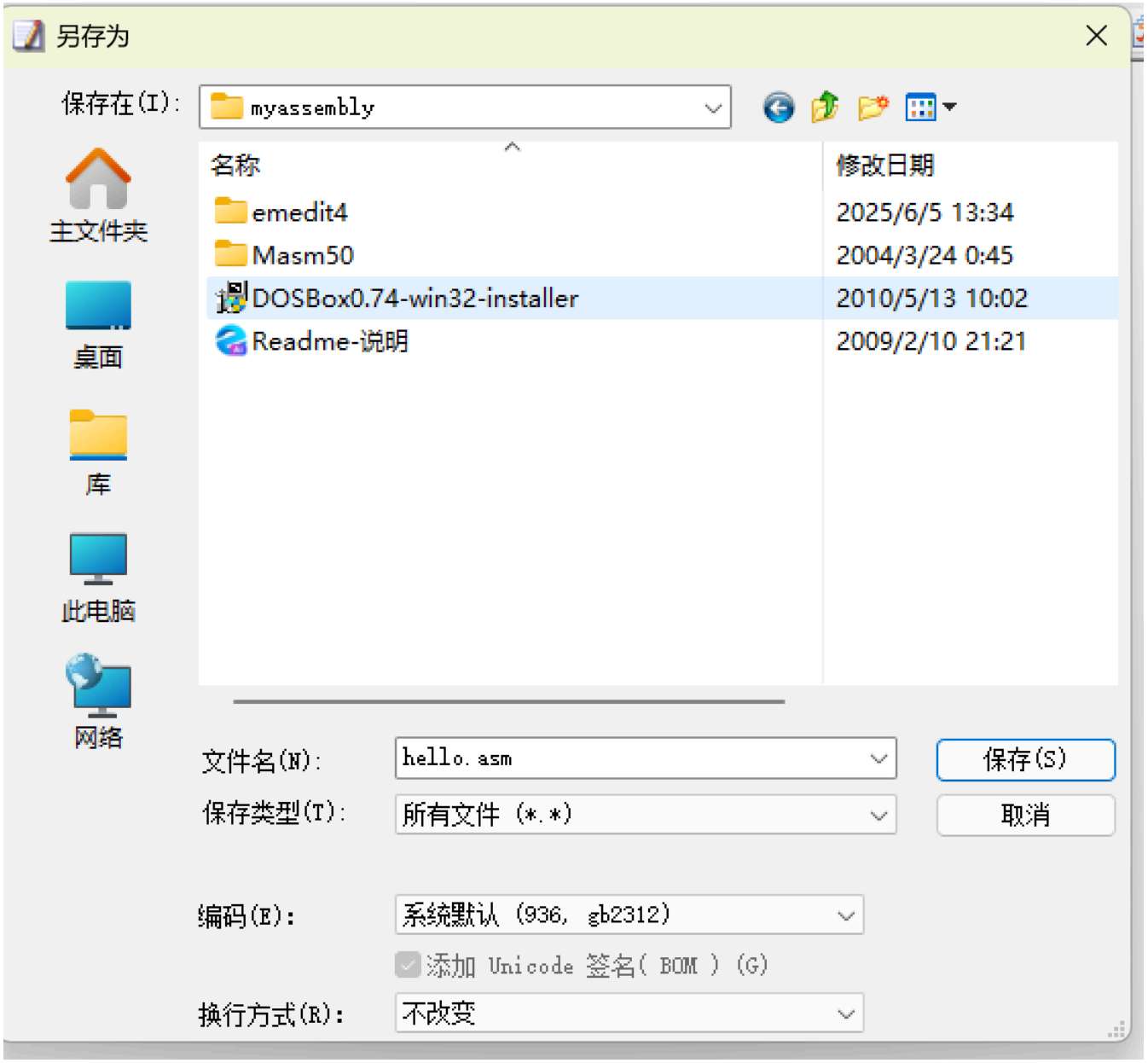
“汇编语言环境搭建与Hello World实验”

> 作业1：搭建环境，用常规方式运行hello world实例，用直接写内存方式运行，理解程序基本结构，和常用寄存器作用。

第一阶段：环境搭建（解决64位系统兼容问题）

1. 在电脑上（比如D盘）新建一个文件夹，命名为 `myassembly`（路径最好全英文，例如 `D:\myassembly`）。
2. 解压软件，将以下文件全部复制到 `D:\myassembly` 文件夹中：
   - `MASM50.rar` 解压出的 `masm.exe` (汇编器) 和 `link.exe` (链接器)。
   - `DOSBox0.74-win32-installer.zip` 解压并安装 DOSBox。
   - **关键一步：** 从网上下载或从32位系统拷贝一个 `debug.exe` 放入 `D:\myassembly`。群文件里没有提供，你需要自己找一下（通常学汇编都会附带这个文件）。
   - 用记事本（或群文件里的 `emedit4.rar` 解压出的编辑器）新建一个文本文件，命名为 `hello.asm`。


无标题 * - EmEditor
文件(F) 编辑(E) 搜索(S) 视图(V) 宏(M) 工具(T) 窗口(W) 帮助(H)
.model small
.data
Hello DB 'Hello world!', 0dh, 0ah, 'S'
.code
START:
MOV AX, @DATA
MOV DS, AX
LEA DX, Hello
MOV AH, 9
INT 21H
MOV AX, 4C00H
INT 21H
END START



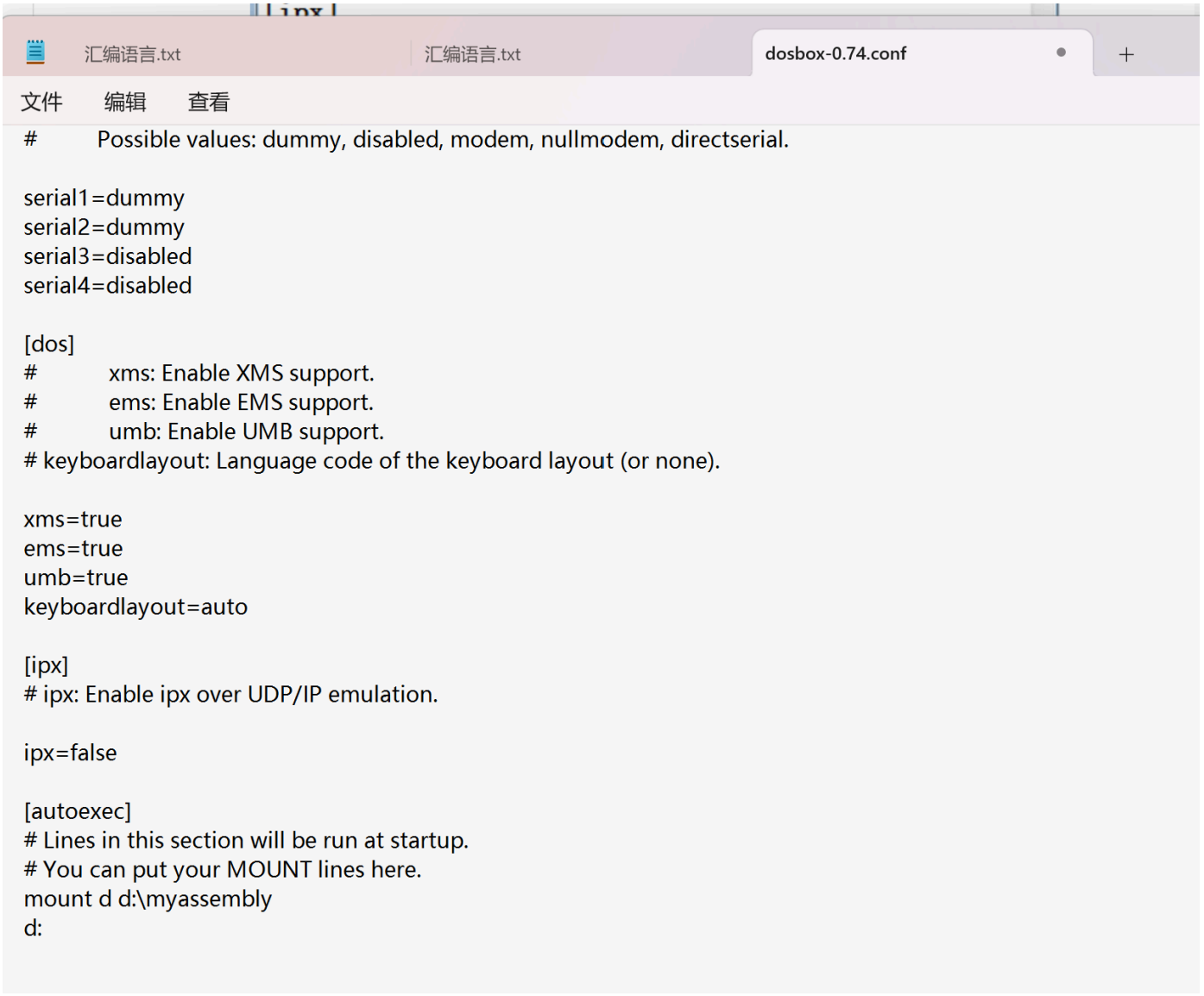
配置 DOSBox 自动挂载：

每次打开DOSBox都要敲命令太麻烦，按照PDF第3-4页的教程设置自动挂载：

1. 打开DOSBox安装目录，双击 `DOSBox 0.74 Options.bat` （或在开始菜单找到Options）。
2. 在弹出的文本文档中，拉到最下面找到 `[autoexec]` 部分。
3. 在末尾添加两行代码（假设你的文件夹在D盘）：

```
代码块
1   mount d d:\myassembly
2   d:
```

4. 保存并关闭文档。



第二阶段：常规方式运行 Hello World

1. 编写汇编源码

打开编辑器，写代码，保存为 `hello.asm`：

代码块

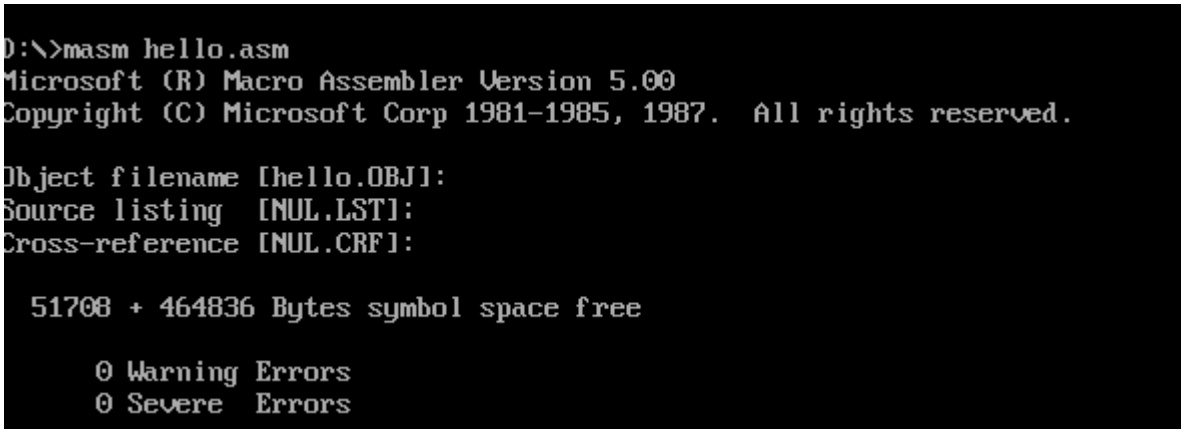
```
1   .model small
2   .data
3       Hello DB 'Hello world!', 0dh, 0ah, '$'
4   .code
5   START:
6       MOV AX, @DATA
7       MOV DS, AX
```

```
    LEA DX, Hello      ; 获取Hello的偏移地址（PDF中重点对比的内容）
    MOV AH, 9          ; 调用DOS 9号功能，显示字符串
    INT 21H
    MOV AX, 4C00H      ; 带返回码退出程序
    INT 21H
END START
```

2. 汇编与链接

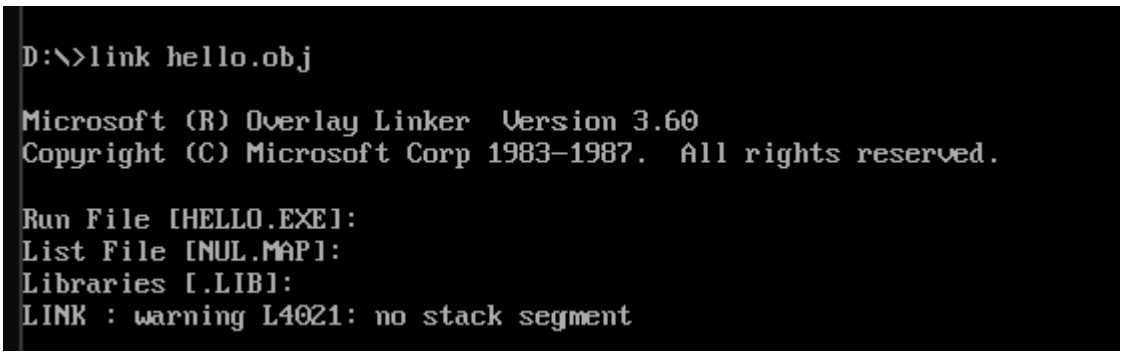
1. 双击打开 **DOSBox**。由于刚才配置了自动挂载，会直接看到 `D:\>` 提示符。
2. 输入以下命令进行**汇编**：

```
masm hello.asm
```

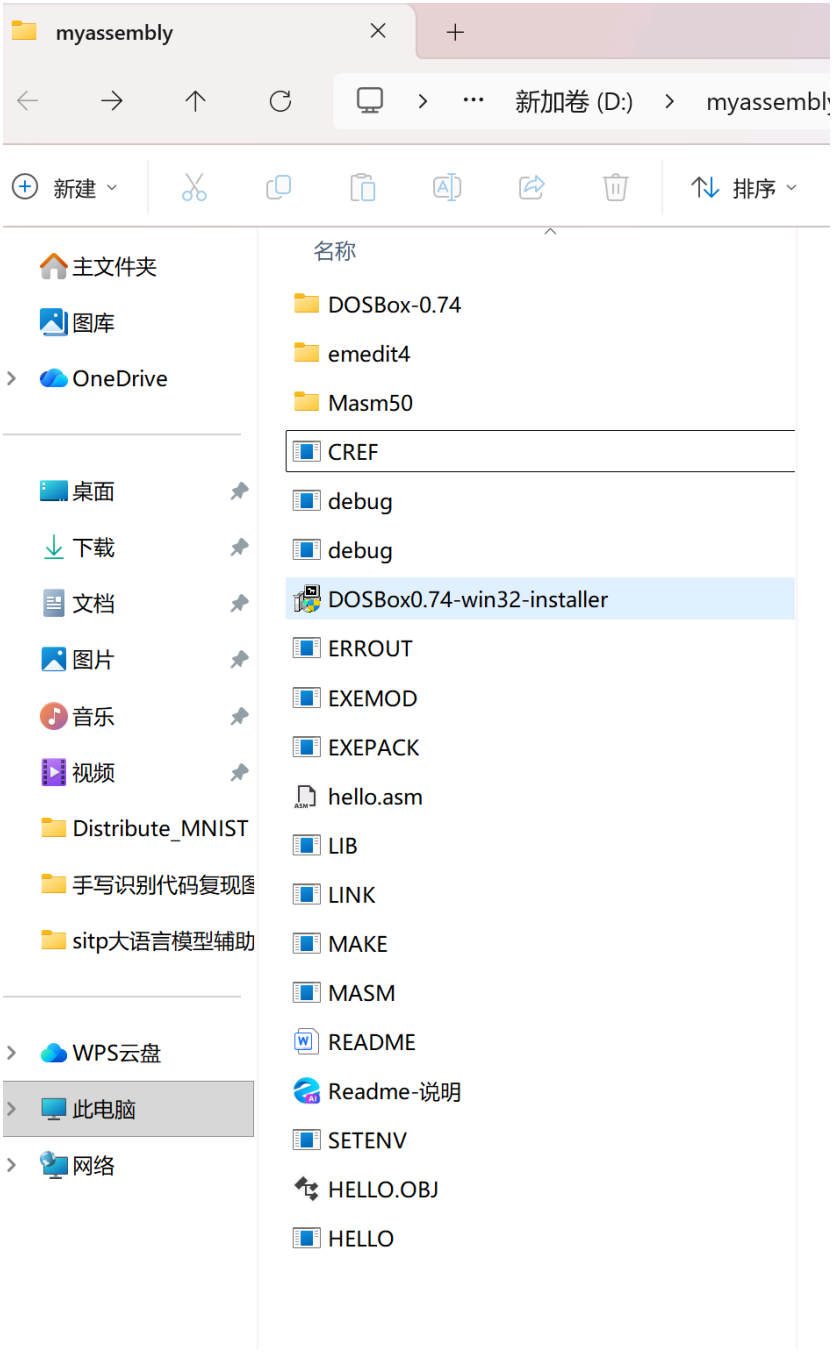


3. 输入以下命令进行**链接**：

```
link hello.obj
```



4. 此时 `D:\myassembly` 文件夹里已经生成了 `hello.exe`。

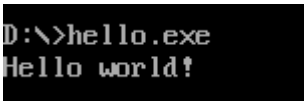


3. 运行程序

在 DOSBox 中输入：

```
代码块
1   hello.exe
```

看到屏幕上输出 `Hello world!`。这证明常规方式实验成功！



第三阶段：直接写内存方式运行与理解寄存器（核心作业）

打开PDF《hello剖析》第2页，对照着做。

```
代码块
1   debug
```

---

① 写数据

输入：

```
代码块
1   e 076a:0
```

然后：

```
代码块
1   48 65 6c 6c 6f 20 77 6f 72 6c 64 21 0d 0a 53 24
```

---

② 检查数据

输入：

```
代码块
1   d 076a:0
```

你应该看到：

```
代码块
1   48 65 6C 6C 6F 20 77 6F 72 6C 64 21 0D 0A 53 24
```

重点看最后：

```
代码块
1   53 24
```

也就是：

```
代码块
1   S$
```

输入：

```
代码块
1   e 076b:0
```

然后：

```
代码块
1   b8 6a 07 8e d8 ba 00 00 b4 09 cd 21 b8 00 4c cd 21
```

输入：

```
代码块
1   r cs
```

输入：

```
代码块
1   076b
```

然后：

```
代码块
```

```
1   r ip
```

输入：

代码块

```
1   0000
```

然后：

代码块

```
1   r ds
```

输入：

代码块

```
1   076a
```

现在输入：

代码块

```
1   r
```

你应该重点看到：

代码块

```
1   CS 076B
2   IP 0000
3   DS 076A
```

然后输入：

代码块

```
1   t
```

第一次执行：

```
代码块
1   MOV AX,076A
```

所以此时：

```
代码块
1   AX = 076A
```

再：

```
代码块
1   t
```

执行：

```
代码块
1   MOV DS,AX
```

所以：

```
代码块
1   DS = 076A
```

再：

```
代码块
1   t
```

执行：

```
代码块
```

```
1   MOV DX,0000
```

所以：

代码块

```
1   DX = 0000
```

这时候最关键的就是：

代码块

```
1   DS = 076A
2   DX = 0000
```

也就是说：

代码块

```
1   DS:DX = 076A:0000
```

再执行：

代码块

```
1   t
```

此时：

代码块

```
1   AH = 09
```

再：

代码块

```
1   t
```

执行：

```
代码块
1   INT 21H
```

应该输出：

```
代码块
1   Hello world!
2   S
```

然后：

```
代码块
1   t
```

执行：

```
代码块
1   MOV AX,4C00H
```

最后：

```
代码块
1   t
```

执行：

```
代码块
1   INT 21H
```

程序退出。


076B:000A CD21 INT 21
-t
Hello world!
SAX=096A BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=076A ES=0724 SS=0724 CS=076B IP=000C NV UP EI PL NZ NA PO NC
076B:000C B8004C MOV AX,4C00
-S_